RUTA FORMATIVA · DESARROLLO BACKEND

Backend y APIs

15 temas para diseñar, implementar y comprobar servicios HTTP

PROPÓSITO

Avanzar desde los fundamentos HTTP hasta una API con contrato, controles de seguridad, persistencia y pruebas.

Recorrido

01-04 Fundamentos

Mensajes HTTP, métodos, estados y diseño de recursos.

05-07 Implementación

Validación, rutas Node.js, errores y diagnóstico.

08-14 Datos y seguridad

Sesiones, CORS/CSRF, SQL, integridad, transacciones, límites y secretos.

15 Entrega

Pruebas de API, contrato OpenAPI y proyecto integrador.

Cómo estudiar cada tema

Lee el objetivo, contrasta los conceptos en las fuentes enlazadas y realiza la práctica. Usa el criterio de comprobación para explicar qué funciona y qué caso límite falta. Los estándares definen el protocolo; OWASP mantiene guías de seguridad, y cada organismo o proyecto documenta sus herramientas. Comprueba la versión y el contexto antes de aplicar una recomendación.

Las recomendaciones de seguridad dependen del contexto del sistema. Esta ruta enseña controles verificables; no fija valores universales de producción.

TEMA 01 / 15

La conversación HTTP

AL TERMINAR

Describir una petición y una respuesta HTTP, e identificar qué parte pertenece al protocolo y cuál al contrato de la aplicación.

Conceptos clave

- Una petición contiene método, destino, cabeceras y, cuando corresponde, contenido.
- Una respuesta contiene código de estado, cabeceras y, salvo casos definidos, una representación.
- HTTP define semántica para los mensajes; la API define los recursos y datos concretos que intercambia.
- Ejemplo y lectura: GET /api/cursos/7 puede responder 200 con una representación JSON; la URI identifica el recurso y el cuerpo describe el dato devuelto.

PRÁCTICA

Dibuja el intercambio para consultar un curso y para crear una matrícula. Anota método, ruta, cabeceras relevantes, cuerpo y respuesta esperada.

COMPRUEBA

Otra persona puede reconstruir ambos intercambios y distinguir petición, respuesta y reglas propias de la aplicación.

Fuentes fiables

RFC 9110 - Semántica HTTP · Node.js - módulo HTTP

TEMA 02 / 15

Métodos: semántica e idempotencia

AL TERMINAR

Elegir un método según el efecto solicitado y razonar sobre reintentos seguros.

Conceptos clave

- GET recupera una representación y debe ser seguro: no se diseña para cambiar el estado solicitado.
- POST solicita procesar el contenido; PUT crea o reemplaza el estado del recurso objetivo; PATCH aplica cambios parciales.
- PUT y DELETE son idempotentes en su efecto previsto: repetir la petición no acumula ese efecto, aunque las respuestas o los registros puedan variar.
- Ejemplo y lectura: repetir PUT /api/cursos/7 con el mismo estado no acumula cambios; repetir POST /api/matriculas puede crear recursos distintos según el contrato.

PRÁCTICA

Diseña operaciones para crear una matrícula, cambiar su estado y eliminarla. Simula dos envíos idénticos de PUT y anota el estado final.

COMPRUEBA

Puedes explicar por qué un método encaja con cada operación y cuándo un cliente puede repetir una petición tras un fallo de red.

Fuentes fiables

RFC 9110 - Semántica HTTP · RFC 5789 - método PATCH

TEMA 03 / 15

Códigos de estado y cabeceras

AL TERMINAR
Comunicar el resultado de una operación con un código y las cabeceras que el cliente necesita.

Conceptos clave
- 2xx indica éxito; 4xx señala un problema de petición o acceso; 5xx señala que el servidor no pudo completar una petición aparentemente válida.
- 201 Created confirma la creación; identifica el recurso primario mediante Location si está presente, o mediante la URI objetivo si no se envía esa cabecera. 204 No Content no incluye una representación.
- 405 Method Not Allowed requiere Allow. Distingue 401 (falta autenticación válida) de 403 (operación no permitida) y usa 404 para un recurso ausente según el contrato.
- 429 puede comunicar que se alcanzó un límite; documenta cómo reintentar, por ejemplo respetando Retry-After si se devuelve.
- Ejemplo y lectura: al crear una matrícula, 201 Created y Location: /api/matriculas/81 permiten reconocer el recurso; 204 significa que no se envía representación.

PRÁCTICA
Completa una tabla para creación, lectura, dato inválido, falta de sesión, recurso inexistente, conflicto y exceso de solicitudes. Añade Location o Allow donde aplique.

COMPRUEBA
Cada caso tiene un resultado coherente con HTTP y el cliente puede decidir qué hacer sin interpretar texto ambiguo.

Fuentes fiables
RFC 9110 - Semántica HTTP · RFC 6585 - 429 Too Many Requests · OWASP API4:2023 - consumo de recursos

TEMA 04 / 15

Recursos, rutas y contrato de API

AL TERMINAR
Diseñar rutas centradas en recursos y describir operaciones de forma consistente.

Conceptos clave
- Usa identificadores estables y rutas comprensibles, por ejemplo /api/cursos/{cursoId}/matriculas.
- El método expresa la operación; evita codificar cambios de estado dentro de un GET.
- OpenAPI puede describir rutas, operaciones, parámetros, cuerpos, respuestas y esquemas para que personas y herramientas compartan el contrato.
- Ejemplo y lectura: GET /api/cursos?activo=true filtra una colección; GET /api/cursos/7 identifica un recurso. OpenAPI debe describir parámetros y respuestas de ambas operaciones.

PRÁCTICA
Define las rutas para consultar cursos, recuperar uno por ID, matricular a una persona y cancelar la matrícula. Especifica parámetros y respuestas.

COMPRUEBA

Dos clientes pueden implementar la API a partir de la descripción sin adivinar nombres, formatos o códigos.

Fuentes fiables

OpenAPI Specification 3.1.0 · RFC 9110 - Semántica HTTP

TEMA 05 / 15

JSON, esquemas y validación

AL TERMINAR

Definir entradas explícitas y rechazarlas en el servidor cuando incumplen el contrato.

Conceptos clave

- Valida tipos, campos requeridos, rangos numéricos, formatos, opciones permitidas y límites de longitud.
- Valida también cantidad de elementos y estructura de objetos anidados; un formulario del navegador no es una barrera de seguridad.
- Limita el tamaño del cuerpo antes de procesarlo. Un esquema OpenAPI ayuda a expresar la forma válida, pero el servidor debe aplicarla.
- Ejemplo y lectura: {"cursoId":7,"email":"ana@example.test"} puede cumplir el esquema; si cursoId llega como texto, el servidor lo rechaza antes de persistir.

PRÁCTICA

Escribe el esquema de una matrícula con cursoId, email y fecha. Añade ejemplos válidos, vacíos, de tipo incorrecto y fuera de rango.

COMPRUEBA

Los casos inválidos se rechazan antes de modificar datos y los mensajes identifican el campo sin devolver información interna.

Fuentes fiables

OWASP - validación de entradas · OpenAPI Specification 3.1.0

TEMA 06 / 15

Node.js: rutas y ciclo de una petición

AL TERMINAR

Relacionar el flujo de una ruta con las interfaces HTTP de bajo nivel de Node.js.

Conceptos clave

- node:http expone objetos IncomingMessage y ServerResponse; permite trabajar con cabeceras y flujos.
- El módulo HTTP no interpreta por sí mismo el contrato JSON de la aplicación: el servidor debe leer y validar el cuerpo.
- Separa resolución de ruta, validación, lógica de negocio, acceso a datos y construcción de respuesta. Mantén acotado el trabajo síncrono.
- Ejemplo y lectura: una petición POST llega como flujo de bytes; la ruta aplica un límite, lee el cuerpo y analiza JSON antes de llamar al caso de uso.

PRÁCTICA
Implementa GET /api/cursos y POST /api/cursos en un servidor de práctica. Rechaza cuerpos demasiado grandes y envía una sola respuesta por petición.

COMPRUEBA
La ruta distingue método y destino, maneja errores de lectura y no bloquea el proceso con trabajo síncrono innecesario.

Fuentes fiables

Node.js - módulo HTTP · Node.js - trabajo asíncrono

TEMA 07 / 15

Errores, registros y diagnóstico

AL TERMINAR
Dar al cliente un error útil y mantener detalles internos en los registros controlados del servidor.

Conceptos clave

- Usa una forma estable para errores, por ejemplo código legible, mensaje breve e identificador de solicitud.
- No devuelvas trazas, consultas, rutas del sistema, credenciales ni detalles de implementación al cliente.
- Registra contexto suficiente para diagnosticar, con cuidado de no guardar contraseñas, tokens ni cuerpos sensibles.
- Ejemplo y lectura: responde 400 con un código estable como JSON_INVALIDO; conserva el identificador de solicitud en el log y omite el cuerpo sensible.

PRÁCTICA
Diseña respuestas para JSON mal formado, error de validación y fallo interno. Comprueba qué ve el cliente y qué información útil conserva el registro.

COMPRUEBA
El cliente recibe una respuesta consistente y el registro permite investigar sin exponer secretos o datos que no hacen falta.

Fuentes fiables

OWASP - errores en API REST · OWASP - registro seguro · Node.js - módulo HTTP

TEMA 08 / 15

Autenticación, autorización y sesiones

AL TERMINAR
Separar la identidad de la persona de sus permisos sobre una operación y un recurso concretos.

Conceptos clave

- Autenticación valida quién presenta la sesión; autorización decide qué acción puede realizar sobre cada objeto.

- Aplica la comprobación de acceso en el servidor para cada recurso indicado por un ID; no confíes en IDs, roles o permisos enviados por el cliente.
- Define caducidad de sesión en el servidor y configura cookies de sesión con atributos apropiados, como HttpOnly, Secure bajo HTTPS y SameSite.
- Ejemplo y lectura: aunque la sesión sea válida, A no puede modificar /api/matriculas/42 si pertenece a B; el servidor comprueba la autorización sobre ese objeto.

PRÁCTICA

Crea dos cuentas y dos matrículas. Prueba si la cuenta A puede leer, cambiar o borrar la matrícula de B alterando el ID.

COMPRUEBA

Cada intento sobre un objeto ajeno se rechaza y una sesión vencida deja de autorizar aunque el navegador conserve una copia de la cookie.

Fuentes fiables

OWASP API1:2023 - autorización por objeto · OWASP - gestión de sesiones

TEMA 09 / 15

CORS y CSRF

AL TERMINAR

Explicar dos controles distintos del navegador para que no se confundan con la autenticación de una API.

Conceptos clave

- CORS regula si un navegador permite que una página de otro origen lea una respuesta; no concede permisos de negocio.
- Las peticiones no simples pueden activar una comprobación previa OPTIONS; para solicitudes con credenciales, el servidor debe permitir explícitamente el origen.
- CSRF intenta inducir al navegador autenticado a enviar una acción no deseada. Usa una defensa adecuada, como token o validación de origen; SameSite es defensa en profundidad.
- Ejemplo y lectura: CORS puede impedir que el navegador lea una respuesta cross-origin, pero no concede acceso al recurso; una cookie automática exige valorar CSRF.

PRÁCTICA

Dibuja una petición same-origin y otra cross-origin con cookie. Marca qué valida CORS, qué valida CSRF y qué permiso sigue comprobando el servidor.

COMPRUEBA

Puedes mostrar un caso donde CORS bloquea la lectura pero no sustituye la autorización ni una defensa CSRF.

Fuentes fiables

WHATWG - Fetch y CORS · OWASP - prevención de CSRF

TEMA 10 / 15

SQL injection y acceso a datos

AL TERMINAR

Mantener los datos de usuario fuera de la sintaxis SQL y limitar el alcance de cada consulta.

Conceptos clave

- Usa consultas parametrizadas para los valores. No unas texto recibido del cliente dentro de la consulta.
- Si una parte estructural no admite parámetros, como una columna de ORDER BY, elige entre opciones fijas permitidas por la aplicación.
- Concede a la cuenta de base de datos los permisos mínimos que necesita y valida los permisos del usuario antes de consultar o cambiar un objeto.
- Ejemplo y lectura: WHERE id = $1 enlaza el valor como dato; una columna de ORDER BY se elige de una lista fija porque el identificador no es un valor enlazable.

PRÁCTICA

Implementa una búsqueda por nombre con parámetros. Prueba comillas y caracteres SQL como datos literales, y revisa que no cambien la consulta.

COMPRUEBA

Los valores externos siempre llegan como datos enlazados y el usuario de la aplicación no tiene permisos innecesarios.

Fuentes fiables

OWASP - prevención de inyección SQL · PostgreSQL - consultas · OWASP API1:2023 - autorización por objeto

TEMA 11 / 15

Modelo de datos e integridad

AL TERMINAR

Expresar reglas de consistencia en el esquema de la base de datos además de validarlas en la aplicación.

Conceptos clave

- Una clave primaria identifica filas; una clave foránea representa una relación y puede impedir referencias inexistentes.
- NOT NULL, UNIQUE y CHECK expresan reglas diferentes: presencia, unicidad y una condición sobre valores.
- Las restricciones de la base protegen la integridad frente a rutas, importaciones o procesos distintos que escriban los mismos datos.
- Ejemplo y lectura: UNIQUE (alumno_id, curso_id) impide repetir la pareja; una clave foránea impide guardar una matrícula para un alumno inexistente.

PRÁCTICA

Modela alumno, curso y matrícula. Impide duplicar una pareja alumno-curso y crea una clave foránea para cada relación.

COMPRUEBA
La base rechaza una matrícula repetida o una referencia a un alumno inexistente, incluso si se omite la validación de interfaz.

Fuentes fiables
PostgreSQL - restricciones

TEMA 12 / 15

Transacciones y consistencia

AL TERMINAR
Agrupar escrituras relacionadas cuando el sistema debe aplicar todas o ninguna.

Conceptos clave
- Una transacción delimita una unidad de trabajo: confirma con COMMIT o descarta con ROLLBACK.
- Usa transacciones cuando un caso de negocio modifica varias filas que deben quedar coherentes entre sí.
- Mantén el alcance claro y breve; maneja errores y define qué respuesta recibe el cliente si la operación no se confirma.
- Ejemplo y lectura: inserta la matrícula y actualiza el contador dentro de la misma transacción; si falla la segunda escritura, ROLLBACK evita conservar solo la primera.

PRÁCTICA
Implementa la matrícula que inserta una relación y actualiza un contador. Fuerza un error en la segunda escritura y comprueba que no quede un estado parcial.

COMPRUEBA
Tras éxito aparecen todos los cambios; tras fallo no queda ninguno de los cambios que pertenecen a la unidad.

Fuentes fiables
PostgreSQL - transacciones

TEMA 13 / 15

Límites, paginación y frecuencia

AL TERMINAR
Acotar trabajo, memoria y coste que una petición puede exigir al servicio.

Conceptos clave
- Define máximos para cuerpo, cadenas, arrays, tamaño de página y operaciones por solicitud.
- Usa paginación y límites del lado del servidor; no permitas que un parámetro controle un conjunto sin tope.
- Aplica límites de frecuencia y tiempos máximos acordes con el endpoint; comunica el límite alcanzado con una respuesta clara.

- Ejemplo y lectura: el servidor acota pageSize=5000 a un máximo documentado; ante exceso de frecuencia puede responder 429 y enviar Retry-After.

PRÁCTICA
Define límites para GET /api/cursos y POST /api/cursos/importar. Prueba una página grande, un array largo y solicitudes repetidas.

COMPRUEBA
Cada límite se valida en servidor, se prueba en sus bordes y no depende de que el cliente use la interfaz oficial.

Fuentes fiables
OWASP API4:2023 - consumo de recursos · OWASP - validación de entradas

TEMA 14 / 15

Configuración y secretos

AL TERMINAR
Separar configuración y credenciales del código y reducir quién puede leerlas o usarlas.

Conceptos clave
- No incluyas claves de API, contraseñas o claves de firma en el repositorio, el JavaScript del navegador ni registros.
- Entrega secretos mediante el mecanismo aprobado para el entorno; limita su alcance, revisa acceso, y define cómo revocarlos o rotarlos.
- Si una credencial se expone, considera que ya no es secreta: revócala o rótala y revisa dónde se copió.
- Ejemplo y lectura: DB_URL se carga en el servidor desde la configuración del entorno; no se envía al navegador ni se registra junto con su valor.

PRÁCTICA
Mueve una credencial ficticia fuera del código, agrega una plantilla sin valores secretos y documenta quién puede acceder y cómo se rota.

COMPRUEBA
El repositorio contiene nombres de variables y pasos de configuración, pero ninguna credencial utilizable.

Fuentes fiables
OWASP - gestión de secretos

TEMA 15 / 15

Pruebas, contrato y proyecto integrador

AL TERMINAR

Comprobar el comportamiento observable de una API y entregar un contrato que otro cliente pueda seguir.

Conceptos clave

- Combina pruebas del caso normal con límites, entradas inválidas, permisos, IDs ajenos, conflictos y fallos de persistencia.
- En Node.js, node:test ofrece un ejecutor integrado; prueba la API mediante sus peticiones y respuestas observables.
- Mantén la descripción OpenAPI alineada con rutas, parámetros, esquemas y códigos que el servidor realmente implementa.
- Ejemplo y lectura: prueba que POST /api/cursos acepte una entrada válida, rechace un email inválido y deniegue una cuenta sin permiso.

PRÁCTICA

Proyecto: API de cursos con CRUD, validación, base relacional, autorización por objeto, límites y errores consistentes. Entrega especificación OpenAPI y una suite de pruebas.

COMPRUEBA

Las pruebas cubren éxito, error y abuso razonable; una persona ajena al código puede ejecutar el contrato y entender cada resultado.

Fuentes fiables

Node.js - test runner · OpenAPI Specification 3.1.0 · RFC 9110 - Semántica HTTP